

May 16, 2018

Ops and Security in a PaaS and Serverless world

Chris Swan (@cpswan)
Fellow, VP, CTO Global Delivery

Chris Swan – Why Me?

@cpswan



Combat Systems Engineer - Royal Navy
Security R&D - Credit Suisse
CTO Security - UBS



CTO - Cohesive Networks
CTO Global Infrastructure Services - CSC
CTO Global Delivery - DXC Technology





*“With a monolith it's easy...
you just pile a bunch of
appliances in front of it”*

Frank Chen
Andreessen Horowitz podcast
‘All about Microservices’
with Adrian Cockcroft
and Martin Casado

<https://a16z.com/2016/09/01/microservices/>



The Audit Paradox

Building Security In



CC photo by WorldSkills

Looks Like This

Message Validation

Message validation is used to protect your service from malformed messages and message parameters. Message schemas can be used to validate incoming messages, and custom validators can be used to validate parameter data before your service consumes it. Do not trust input from any source that the client can influence, such as cookies, headers, IP address, or the content of messages sent to your service. Do trust input from a database, the file system, or anything else outside the trust boundary of your service. Use message schemas and data validators to check for format, range, length, and type. Do not rely on client-side validation; make all security decisions based on server-side validation.

Consider the following guidelines:

- **Do not trust input.**
- **Verify the message payload against a schema.**
- **Verify the message size, content, and character sets.**
- **Filter, scrub, and reject input and output before additional processing.**

Do Not Trust Input

An attacker passing malicious input can attempt SQL injection, cross-site scripting, and other injection attacks that aim to exploit your application's vulnerabilities. Check for known good data and constrain input by validating it for type, length, format, and range.

Verify the Message Payload Against a Schema

If you need to validate parameters, message contracts, or data contracts passed to operations, use schemas to validate the incoming message. Schemas provide a wide range of input validation without the need for custom code or validation routines.

Verify the Message Size, Content, and Character Sets

Validate incoming messages to ensure that they match your expectations regarding size, content, and character encoding. If a message is much larger than expected or contains encoding other than what your service expects, you may be under attack. Reject the message and log the occurrence so that auditing can determine if an attack is underway.

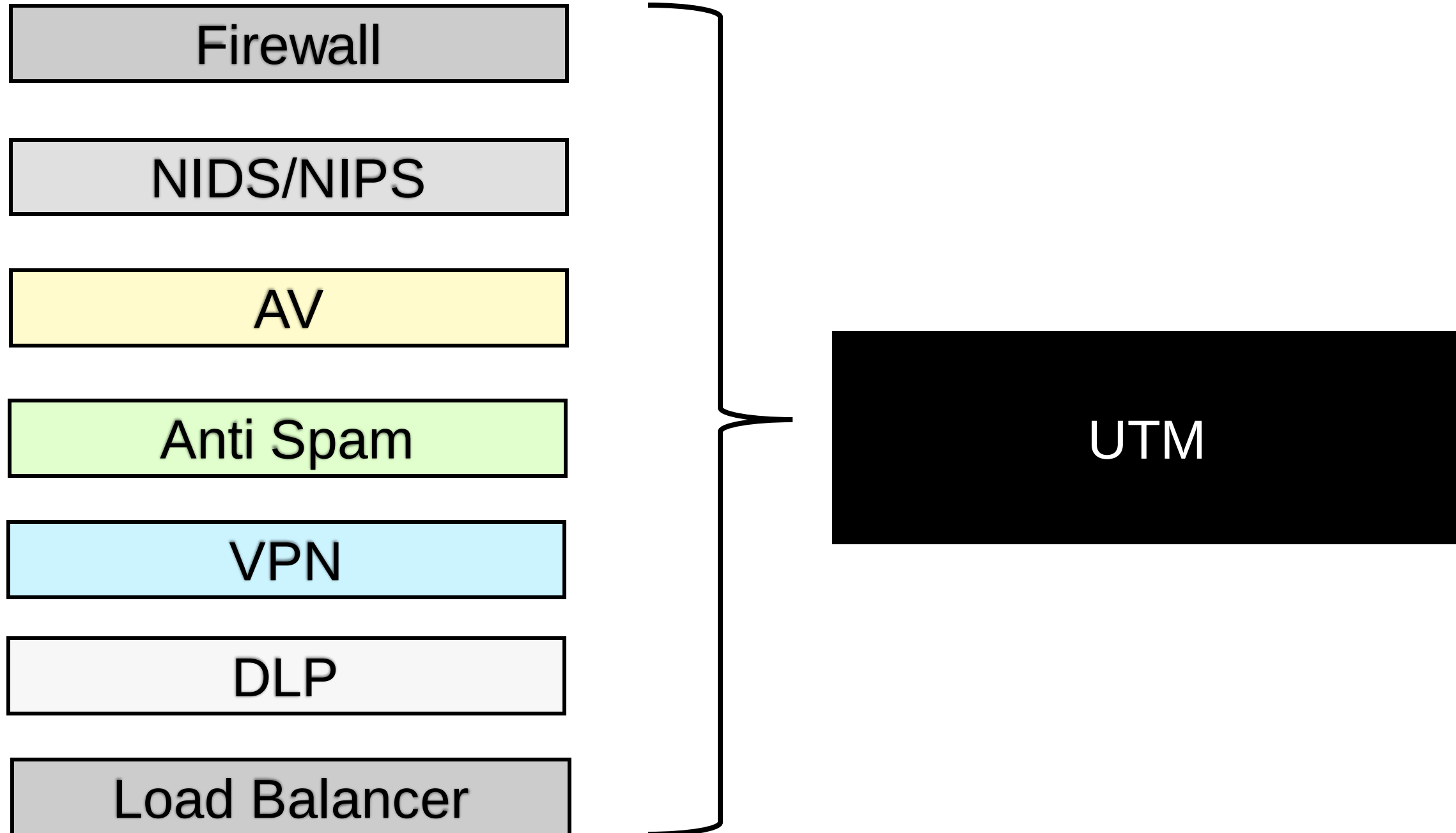
Bolting Security On



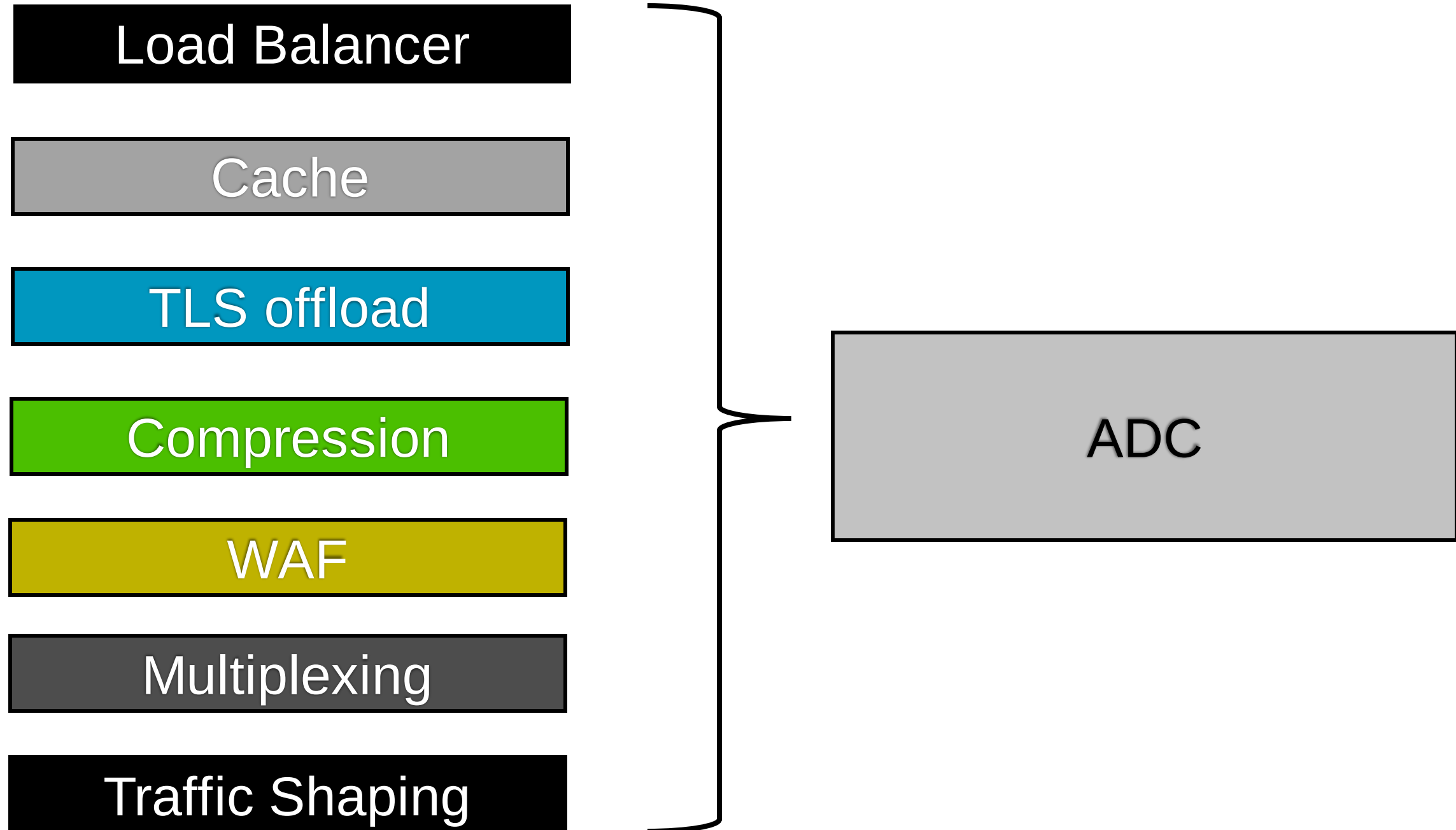
Looks Like This



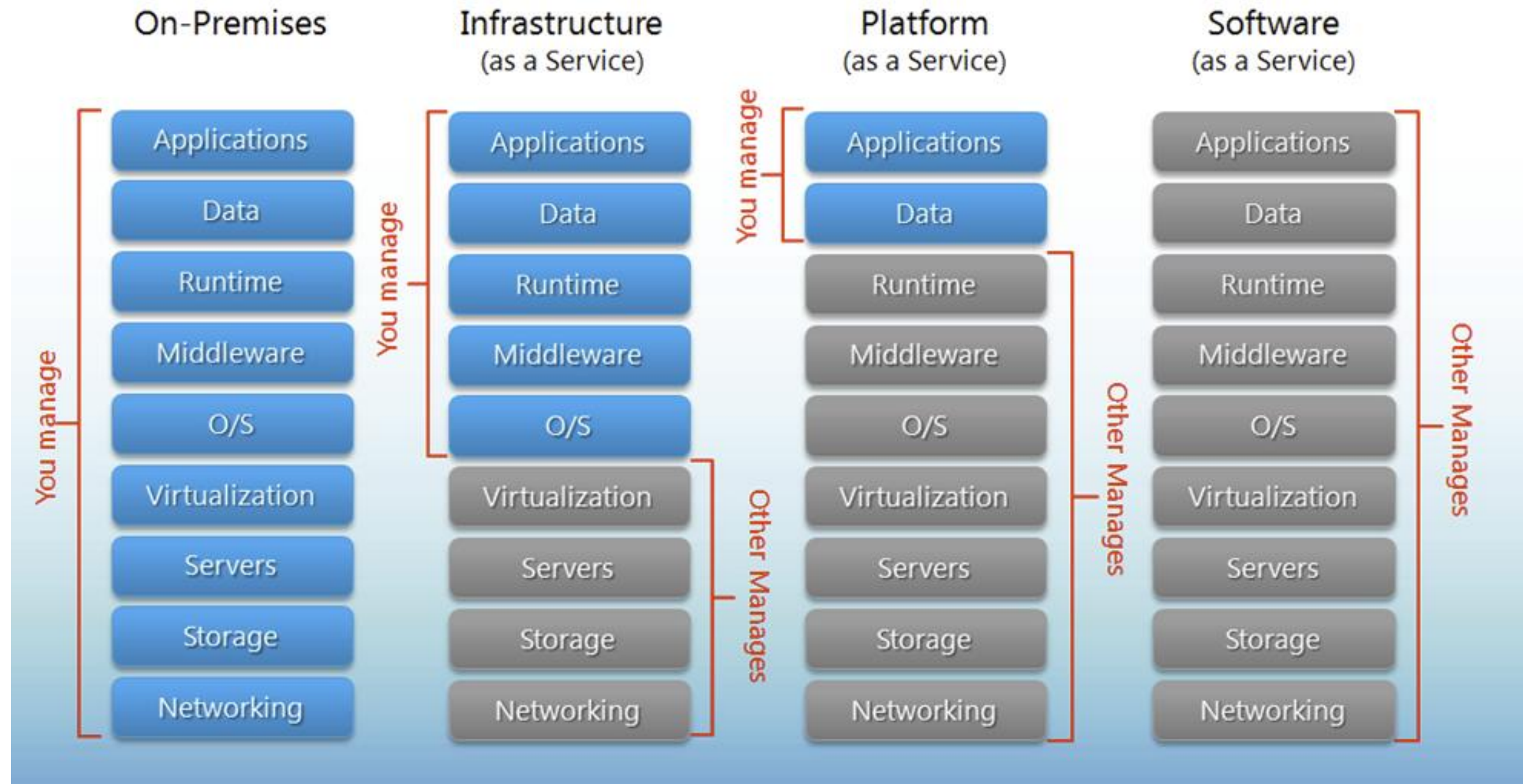
Unified Threat Management



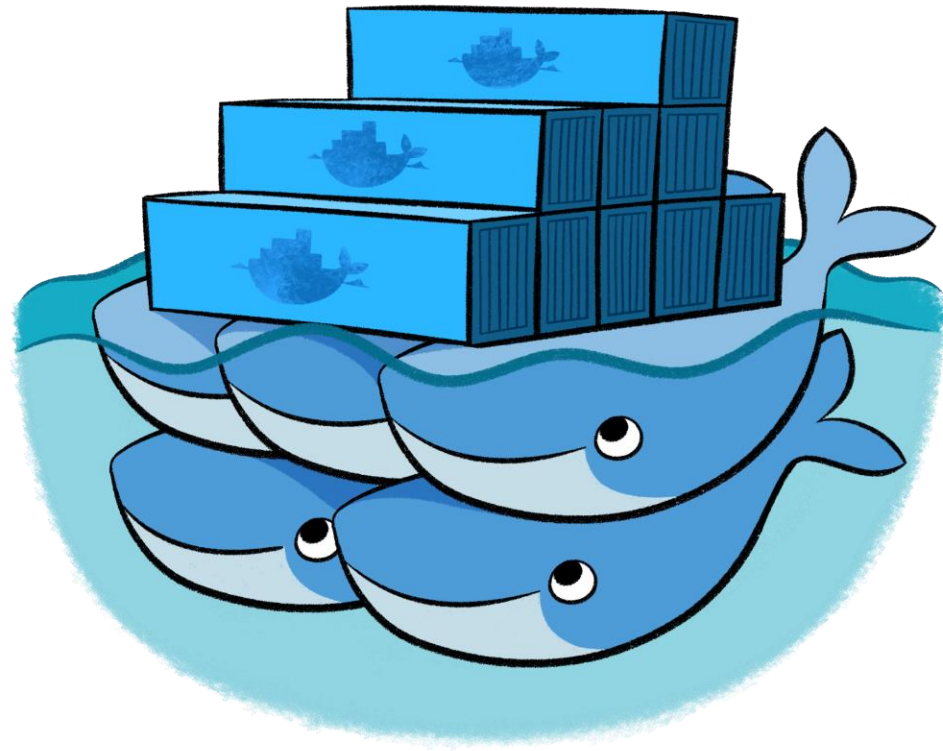
Application Delivery Controllers



PaaS Gives Us the Chance to 'Bolt In'



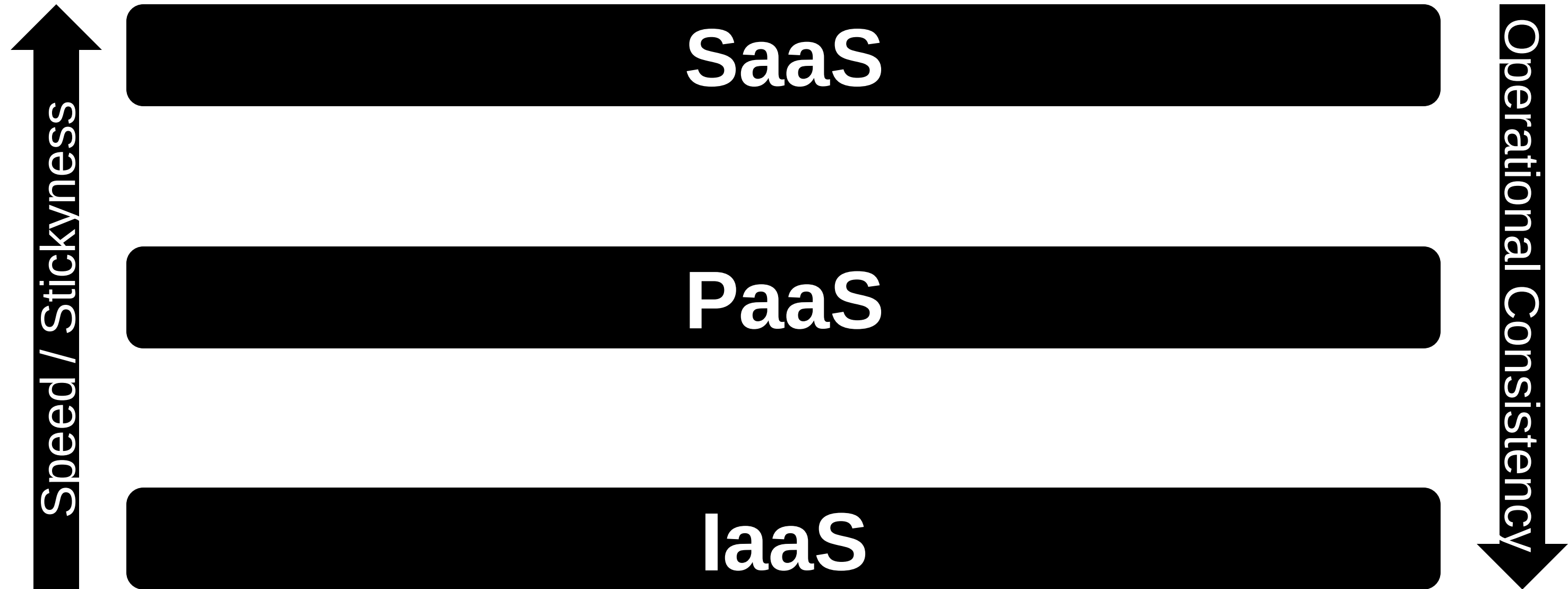
But Docker Adoption Shows a Movement Against Opinionated Platforms



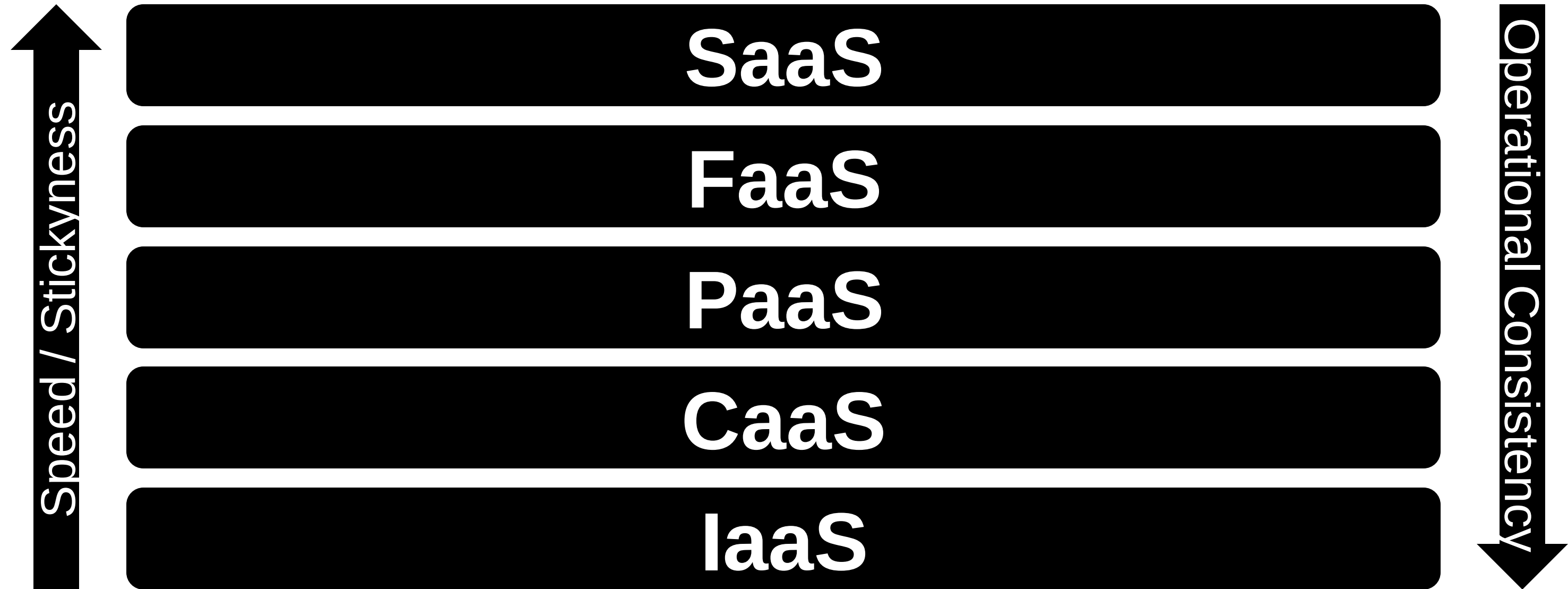


FaaS and CaaS

The NIST Model for Cloud Services



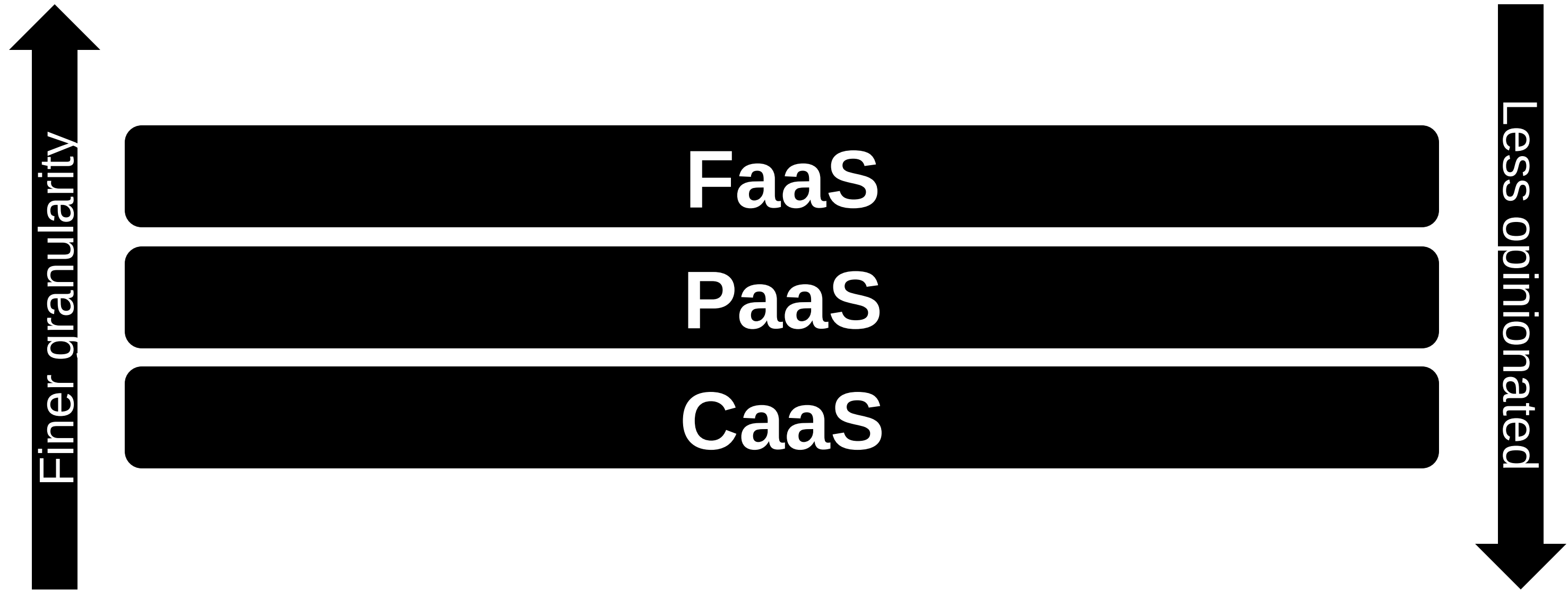
More Choices Have Emerged



And 'Cloud' Doesn't Have to Mean 'Public'



Zooming in Around PaaS and Thinking About Deployment Granularity and Platform 'Opinion'



This Isn't My Container Security Talk...



So Let's Leave CaaS For Another Time

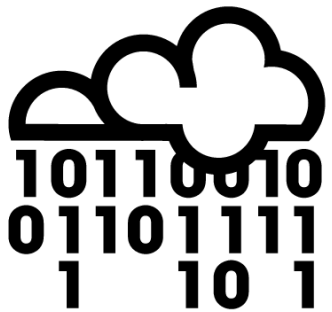
2015 ToDo: SecDevOps – Are We There Yet?



**APIs are necessary but not sufficient:
Need to have them integrated into the overall system**



**Control metadata (and its mutability):
Must be visible and understandable**



**Security events need to be captured:
Then turned into something humans can action**

If a Security Event Happens and It Isn't Monitored





‘LessOps

#define Serverless (aka Functions as a Service – FaaS)



Want to do CaaS and FaaS Together?

FaaS on Kubernetes

<https://github.com/cpswan/FaaSonK8s>

1. Azure Functions Runtime
2. Fission
3. Fn
4. Gestalt
5. IronFunctions
6. Kubeless
7. OpenFaaS
8. OpenWhisk
9. Nuclio
10. Riff
11. VMware Dispatch



#define Operations

Provisioning

Config Management

Monitoring

Logging

Or as Charity Majors @mipsytipsey Put It

“Operations is the constellation of your org's technical skills, practices, and cultural values around designing, building and maintaining systems, shipping software, and solving problems with technology.”



#define DevOps

The operational practices that have co-evolved with IaaS

or

Flow

Feedback

Continuous Learning by Experimentation





#define NoOps

~~Provisioning~~

~~Config Management~~

~~Monitoring~~

~~Logging~~



No Server $\neq$ No Ops



#define LessOps

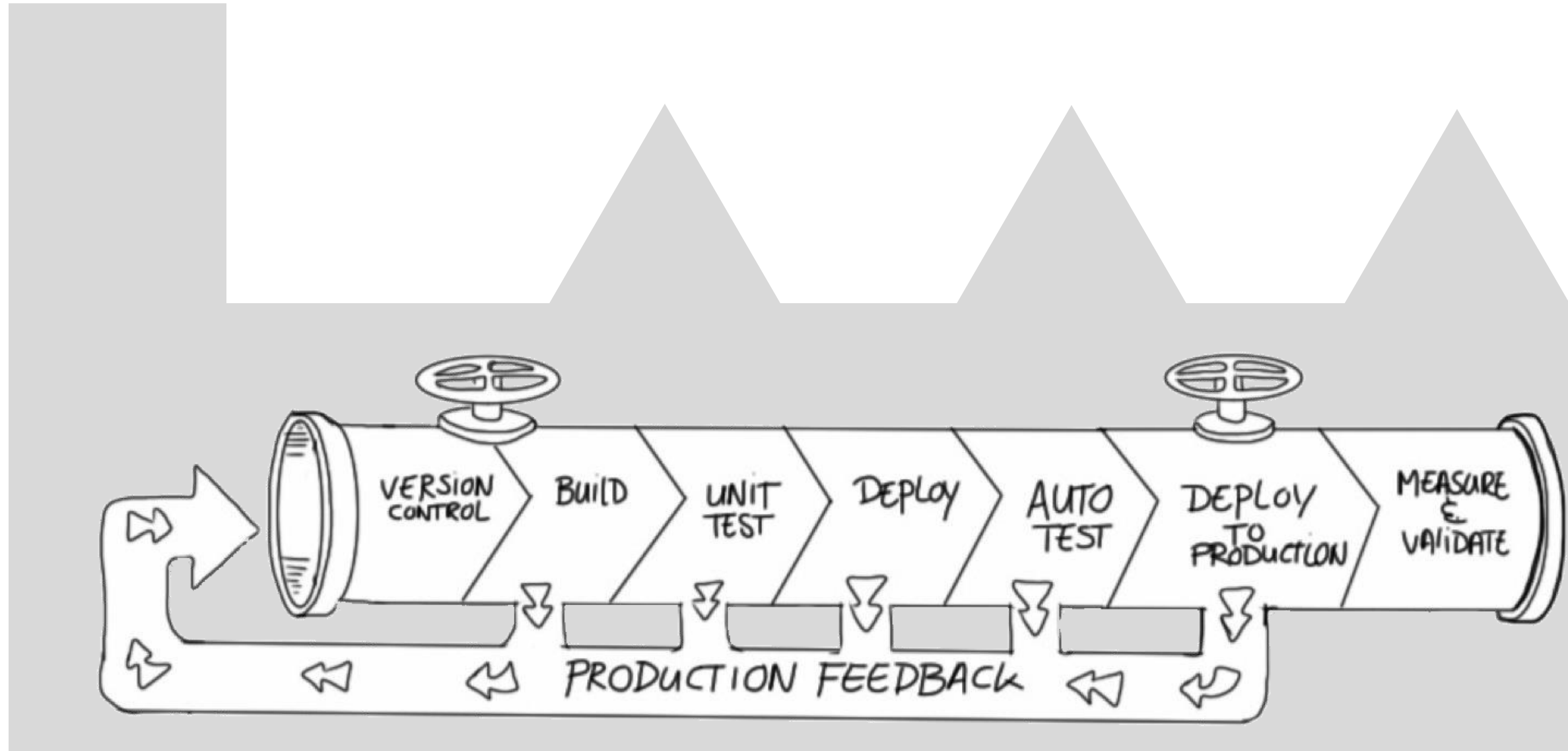
Provisioning aaS

Config Management < will be 'code', won't go away

Monitoring aaS

Logging aaS

Factories for Continuous Delivery Pipelines



We're not there yet



By Alan Manson - Flickr: Yellow Canary (*Serinus flaviventris*), CC BY-SA 2.0
<https://commons.wikimedia.org/w/index.php?curid=17379213>

Javascript hello world

```
exports.myHandler = function(event, context, callback) {  
  callback(null, "Hello World");  
}
```

Gareth Rushgrove



Some required configuration...

```
variable "aws_region" {  
    description = "AWS Region Lambda function is deployed to"  
}  
variable "apex_environment" {  
    description = "Apex configured environment. Auto provided by 'apex infra'"  
}  
variable "apex_function_role" {  
    description = "Provisioned Lambda Role ARN via Apex. Auto provided by 'apex'"  
}  
variable "apex_function_hub" {  
    description = "Provisioned function 'hub' ARN information. Auto provided by"  
}  
variable "apex_function_hub_name" {  
    description = "Provisioned function 'hub' name information. Auto provided by"  
}
```

Gareth Rushgrove



...some more required configuration

```
resource "aws_cloudwatch_event_rule" "every_five_minutes" {  
  name = "every-five-minutes"  
  description = "Fires every five minutes"  
  schedule_expression = "rate(5 minutes)"  
}  
  
resource "aws_cloudwatch_event_target" "check_hub_every_five_minutes" {  
  rule = "${aws_cloudwatch_event_rule.every_five_minutes.name}"  
  target_id = "${var.apex_function_hub_name}"  
  arn = "${var.apex_function_hub}"  
}  
  
resource "aws_lambda_permission" "allow_cloudwatch_to_call_hub" {  
  statement_id = "AllowExecutionFromCloudWatch"  
  action = "lambda:InvokeFunction"  
  function_name = "${var.apex_function_hub_name}"  
  principal = "events.amazonaws.com"
```

Gareth Rushgrove





(barely) 3 lines of Javascript

32 lines of config

A real world example from Serverless



Update: Part two of this series shows how to [expire old snapshots](#).

Backups aren't usually the first thing that comes to mind when you hear about a new service. After the Lambda blog announced [task scheduling](#), I

Gareth Rushgrove



and more different configuration

```
"EbsBackupSnapper": {  
  "Type": "AWS::Lambda::Function",  
  "DependsOn": [  
    "EbsBackupExecutionRole",  
    "EbsBackupExecutionPolicy"  
  ],  
  "Properties": {  
    "Code": {  
      "S3Bucket": "demos.serverlesscode.com",  
      "S3Key": "2015-11-ebs-snapshots/ebs-snapper.zip"  
    },  
    "Role": {  
      "Fn::GetAtt": ["EbsBackupExecutionRole", "Arn"]  
    },  
    "Timeout": 60,  
    "Handler": "lambda_function.lambda_handler",  
    "Runtime": "python2.7",  
  },  
}
```





33 lines of Python

100 lines of config



What's all that config for?

- **Identity and Access Management**
 - Policies
 - Roles
- **API Gateway**
- **Monitoring**
 - Event Targets
 - Event Rules

Hello World with the Serverless Framework

```
# Install serverless globally
$ npm install serverless -g

# Login to your Serverless account
$ serverless login

# Create a serverless function
$ serverless create --template hello-world

# Deploy to cloud provider
$ serverless deploy

# Function deployed! Trigger with live url
$ http://xyz.amazonaws.com/hello-world
```

AWS CodeStar

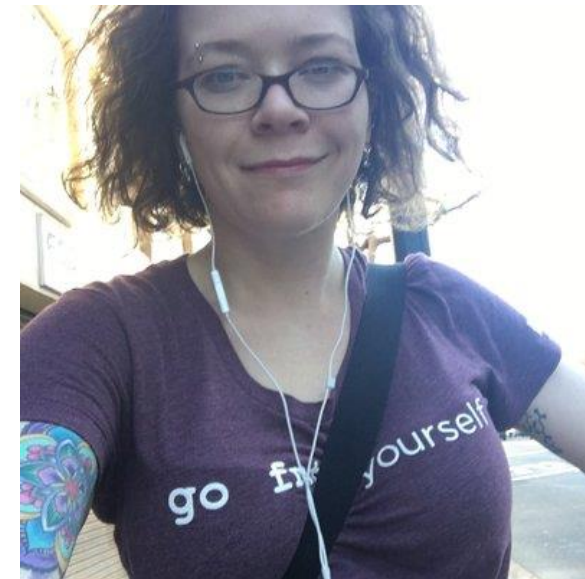
The screenshot shows the AWS CodeStar console interface. At the top, the browser address bar displays the URL `https://console.aws.amazon.com/codestar/home?region=us-east-1#/quickstart`. The navigation bar includes the AWS logo, 'Services', 'Resource Groups', and a user profile for 'Chris Swan'. Below the navigation bar, a breadcrumb trail shows 'AWS CodeStar' and 'Create project'. A progress indicator at the top center shows three steps: 'Select template' (active), 'Set up tools', and 'Start coding'. The main content area is titled 'Choose a project template' and includes a sub-header 'Start a new software project on AWS in minutes using a project template. [Help me choose](#)'. On the left, a sidebar contains filters for 'Application category' (Web application selected), 'Programming languages' (Node.js selected), and 'AWS services' (AWS Lambda selected). The main area displays a 'Node.js' template card with options for 'Web application' and 'AWS Lambda (running serverless)'.

State management

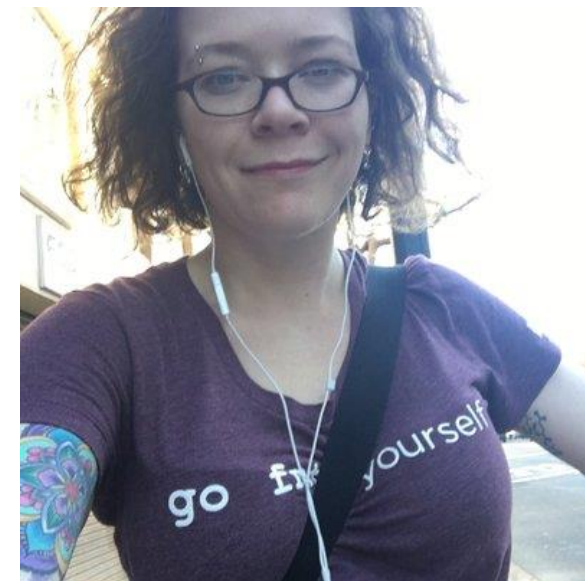


The twelve-factor methodology can be applied to apps written in any programming language, and which use any combination of backing services (database, queue, memory cache, etc).

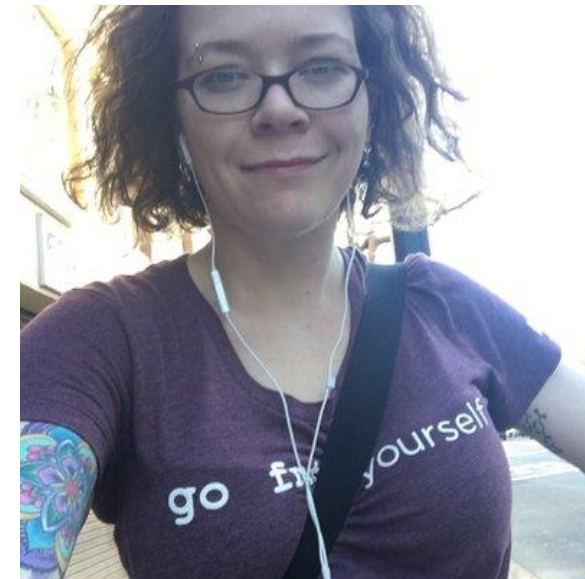
***“The service will protect itself
(at your expense)”***



***“Own the critical path
(and keep it short)”***



***“Understand the tech
and its dependencies”***



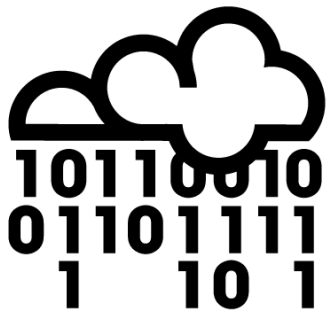
2015 ToDo: SecDevOps – Let's review



**APIs are necessary but not sufficient:
Need to have them integrated into the overall system**



**Control metadata (and its mutability):
Must be visible and understandable**



**Security events need to be captured:
Then turned into something humans can action**

With thanks to:



Charity Majors
@mipsytipsy



Patrick Debois
@patrickdebois



Rafal Gancarz
@RafalGancarz



Paul Johnston
@PaulDJohnston



Questions?